A Project Report
On

DEEPFAKE AUDIO DETECTION

BY

ANVAY SAWAI 2022A3PS0701H
M. HRUSHIKESH 2022AAPS0412H
SHASHWAT PANDEY 2022A71299H
PRADYUMNA UDUPA 2022AAPS0475H

Under the supervision of
DR. PARESH SAXENA

SUBMITTED IN FULFILLMENT OF THE REQUIREMENTS OF MACHINE LEARNING (BITS F464)

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI (RAJASTHAN)
HYDERABAD CAMPUS
(APRIL 2025)

FINAL KAGGLE CODE LINK
https://www.kaggle.com/code/gixo95/deepfake-audio-detection-code

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to Dr. Paresh Saxena for their invaluable guidance, support, and encouragement throughout this project. Their expertise and dedication have been instrumental in overcoming challenges we faced while working on this project. We deeply appreciate their willingness to share their knowledge and take the time out of their busy schedules to assist us.

We are grateful to Birla Institute of Technology and Science – Pilani, Hyderabad Campus for providing us with this wonderful opportunity to work on this project. We are grateful for the necessary resources and the conducive environment that we were provided with for this project.

Lastly, we would like to thank our friends, family and teachers for their constant support and motivation that helped us immensely while working on this project.

ABSTRACT

With advancing artificial intelligence, deepfake audio, artificial, or manipulated speech is a greater security and authenticity risk. For our project, we implemented a deepfake audio detection system based on the research paper "Deepfake Audio Detection via MFCC Features Using Machine Learning." The emphasis was on extracting Mel-Frequency Cepstral Coefficients (MFCC) features from audio samples and using different machine learning and deep learning models for classification.

We used the Fake-or-Real (FoR) dataset with four subsets: for-2sec, for-norm, for-rerec, and for-original. Data preprocessing involved cleaning up corrupted files, normalization of the audio features, and zero-padding for short signals. Feature extraction was centered on MFCC with other spectral features like roll-off and centroid, and then PCA for dimensionality reduction.

Machine learning models like SVM, Random Forest, Gradient Boosting, and MLP were trained and tested. MLP model achieved maximum accuracies in for-2sec and for-rerec datasets with and SVM achieved best on for-norm. For tough for-original dataset, a deep learning model of VGG-16 was fine-tuned based on MFCC spectrogram images and achieved improved testing accuracy improving over existing techniques.

The success of the project proves that machine learning along with MFCC-based feature engineering is capable of providing effective detection features and deep learning models such as VGG-16 can be helpful for processing intricate data.

CONTENTS

INTRODUCTION

In the past few years, the diffusion and proliferation of deepfake technologies have caused formidable challenges to information security, digital trust, and social stability. Deepfakes, produced by advanced artificial intelligence (AI) and deep learning technologies, can be used to create realistic yet artificial multimedia data, like videos, images, text, and audio. Though significant efforts have been dedicated to visual deepfake detection, audio deepfake detection is comparatively less studied even though its seriousness is high. The ability to create high-fidelity human-like voices is threatening in a pivotal manner, ranging from identity theft and financial scams to political disinformation and social engineering attacks.

Audio deepfakes are generally synthesized with state-of-the-art text-to-speech (TTS) and voice conversion algorithms like WaveNet, Tacotron, and DeepVoice. These frameworks can mimic human speech's singularities like tone, pitch, and speech rhythm, rendering even humans and mainstream machine learning setups unable to reliably differentiate between natural and artificial audio. With increasing use of voice-based authentication protocols in banking, telephony, and personal computers, creating highly effective methods to detect deepfake audio has assumed a pressing position in research efforts.

The present work is based on the implementation and extension of the methodology described in the paper "Deepfake Audio Detection via MFCC Features Using Machine Learning." The primary focus is placed on the extraction of Mel-Frequency Cepstral Coefficients (MFCC), a standard feature set utilized in speech recognition, to acquire valuable acoustic information that can differentiate between human-speaking synthetic and natural speech. Certain additional spectral and temporal features were also utilized to enhance the performance of the model. Fake-or-Real (FoR) dataset, developed very recently as a benchmarking dataset for the detection of deepfake audio, has been employed in this study. It has four distinct subsets (for-2sec, for-norm, for-rerec, and for-original) with diverse conditions of recording, lengths of duration, and noise amplitudes. Rigorous data preprocessing was carried out with the deletion of duplicate and zero-bit audio files, standardization of sampling rates, and padding shorter audio samples to such an extent that dataset uniformity is created. Feature extraction based on MFCC and spectral roll-off, centroid, contrast, bandwidth, and energy was utilized to generate a large number of features.

Various supervised machine learning algorithms, such as Support Vector Machine (SVM), Random Forest, Multi-Layer Perceptron (MLP), and Gradient Boosting, were tried and trained. SVM classifier gave the best result on the for-2sec and for-rerec datasets, and the best result was obtained using Gradient Boosting on the for-norm subset. Since the for-original dataset is heterogeneous and complex, a deep learning-based transfer learning method was used. VGG-16 model pre-trained on ImageNet was fine-tuned on MFCC spectrogram representations, and the testing accuracy obtained was well above baseline models.

This project thus provides a significant contribution to the emerging field of audio forensics by providing an effective system to identify deepfake audio. As a future extension, the integration of more advanced architectures, such as Transformer-based models, and further real-world samples added to the dataset to increase the reliability and scalability of the system is possible.

RESEARCH PAPER IMPLEMENTATION

The project was based on the research paper titled *"Deepfake Audio Detection via MFCC Features Using Machine Learning"*, published in the IEEE Conference. It proposes the use of Mel-Frequency Cepstral Coefficients (MFCC) for feature extraction and traditional machine learning classifiers for deepfake audio detection. We closely followed and extended these methods by implementing multiple machine learning algorithms for detection and evaluation across multiple datasets.

Dataset Used:

We used the **Fake-or-Real (FoR)** dataset, which is a collection of more than 195,000 utterances from real humans and computer generated speech. The dataset can be used to train classifiers to detect synthetic speech. The dataset aggregates data from the latest TTS solutions (such as Deep Voice 3 and Google Wavenet TTS) as well as a variety of real human speech, including the Arctic Dataset (http://festvox.org/cmu_arctic/), LJSpeech Dataset (https://keithito.com/LJ-Speech-Dataset/), VoxForge Dataset (http://www.voxforge.org) and our own speech recordings. It contains four subsets:

- **for-original**: It contains the files as collected from the speech sources, without any modification (balanced version).
- **for-norm**: It contains the same files, but balanced in terms of gender and class and normalized in terms of sample rate, volume and number of channels.
- **for-2seconds**: It is based on the second one, but with the files truncated at 2 seconds.
- **for-rerec**: Re-recorded audio samples introducing real-world noise.

Each dataset contained both real and deepfake audio samples which enabled us to conduct a rigorous evaluation across different audio qualities and conditions.

Preprocessing:

- **Standardization**: All audio files were standardized to a common sampling rate.
- **Noise Cleaning**: Corrupted or silent audio files were identified and removed.
- **Zero Padding**: To ensure uniform input lengths, short audio clips were zero-padded.
- **Feature Extraction**:
  - **MFCC Features** were extracted from each audio file.
  - Additional **spectral features** such as spectral roll-off, centroid, contrast, and bandwidth were also computed to enrich the feature set for the Deep Learning Model.

Model Training:

We trained four machine learning models across each dataset:

- **Random Forest Classifier**
- **Support Vector Machine (SVM)**
- **Multi-Layer Perceptron (MLP)**
- **Extreme Gradient Boosting (XGBoost)**

Each model was trained using the extracted MFCC and spectral features. Model evaluation was based on key metrics such as **accuracy** and **precision**.

Summary of Results:

Dataset	Random Forest Accuracy	SVM Accuracy	MLP Accuracy	XGBoost Accuracy
2seconds	55.14%	57.72%	60.47%	59.65%
norm	77.34%	90.89%	90.74%	76.56%
original	71.17%	89.21%	95.45%	70.34%
rerec	72.91%	78.19%	81.25%	74.14%

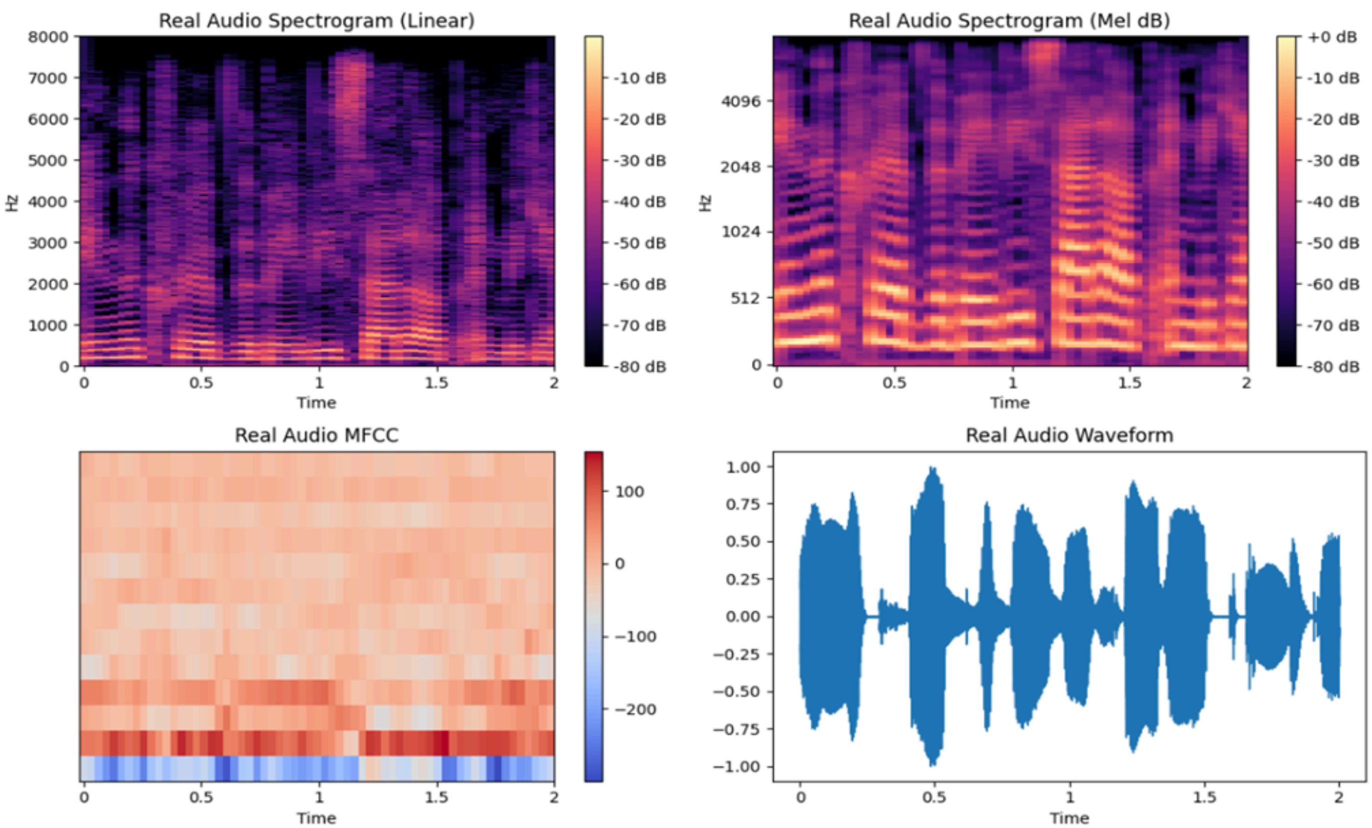
Key Observations:

- The **MLP model** performed best overall, achieving the highest accuracy of **95.45%** on the challenging **original** dataset.
- The **SVM model** showed strong performance on **norm** and **original** datasets as well.
- **Random Forest** and **XGBoost** performed reasonably well but lagged slightly behind the MLP and SVM, especially on complex datasets.
- Performance across models was generally lower on the **2seconds** dataset due to the shorter audio length, making deepfake artifacts harder to detect.

Through the implementation of the methods proposed in the original paper, we successfully developed a robust pipeline for deepfake audio detection. The results affirm the effectiveness of MFCC-based feature extraction coupled with classical machine learning models.

VISUALIZATIONS

The figures below represent the **comparative visualizations** of real and fake audio samples through four key perspectives: linear spectrogram, mel spectrogram (in decibels), MFCCs (Mel-Frequency Cepstral Coefficients), and waveform analysis. These visual representations provide crucial insight into the underlying structure of real and synthesized (deepfake) audio signals.



Spectrogram (Linear and Mel dB)

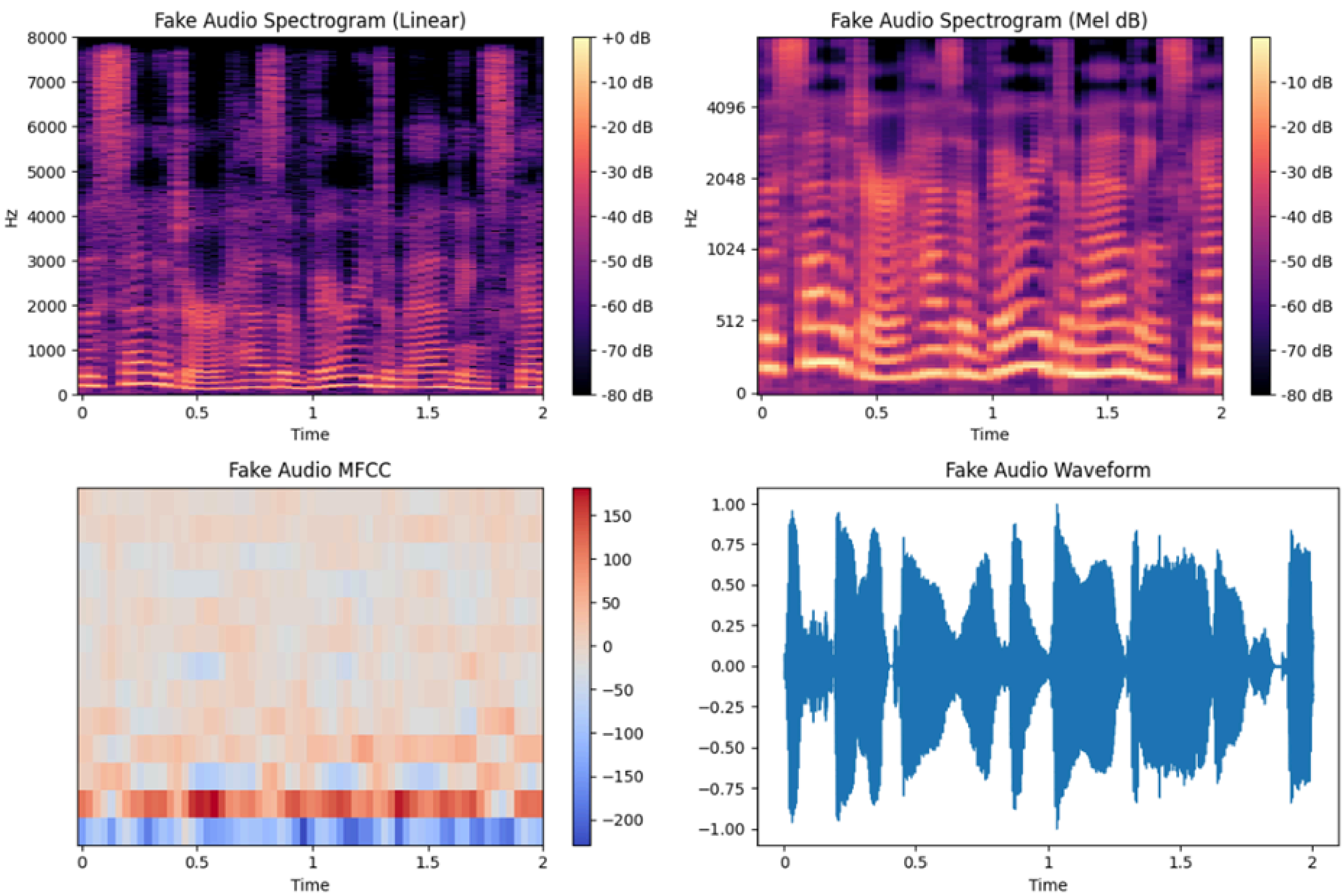
- The **linear spectrograms** for both real and fake audio display the intensity of frequency components over time. The real audio shows more natural and smoother transitions in frequency bands, while the fake audio presents more **discontinuities** and **unnatural gaps**, indicating potential synthesis artifacts.
- The **Mel spectrogram** further emphasizes perceptually-relevant frequency content. The real sample exhibits **harmonic richness** and **consistent energy distribution**, whereas the fake audio's mel spectrogram reveals **irregular energy bands** and **less harmonic structure**, which is a common characteristic of generated or synthesized speech.

MFCCs (Mel-Frequency Cepstral Coefficients)

- MFCCs are widely used for feature extraction in speech-related machine learning tasks. The MFCC plot of real audio reveals a **more structured and smooth pattern**, aligning with the expected phonetic structure of natural speech.
- On the contrary, the fake audio MFCCs display **scattered and inconsistent patterns**, highlighting the deviation from natural articulation. These variations help the model distinguish between authentic and synthetic audio effectively.

Waveform

- The waveform plots for real and fake audio show further distinction. Real audio typically has a **more rhythmic and smooth envelope**, whereas the fake audio waveform exhibits **sudden spikes and inconsistent amplitude variations**, which may arise from synthesis errors or compression during generation.



These visualizations validate the findings from the referenced IEEE paper. The visual analysis supports the effectiveness of MFCC and spectrogram-based features in distinguishing real and deepfake audio. Notably, the irregularities in frequency components and MFCC patterns in fake samples provide strong cues for machine learning models to learn and classify audio authenticity with high accuracy.

Model Performance Evaluation

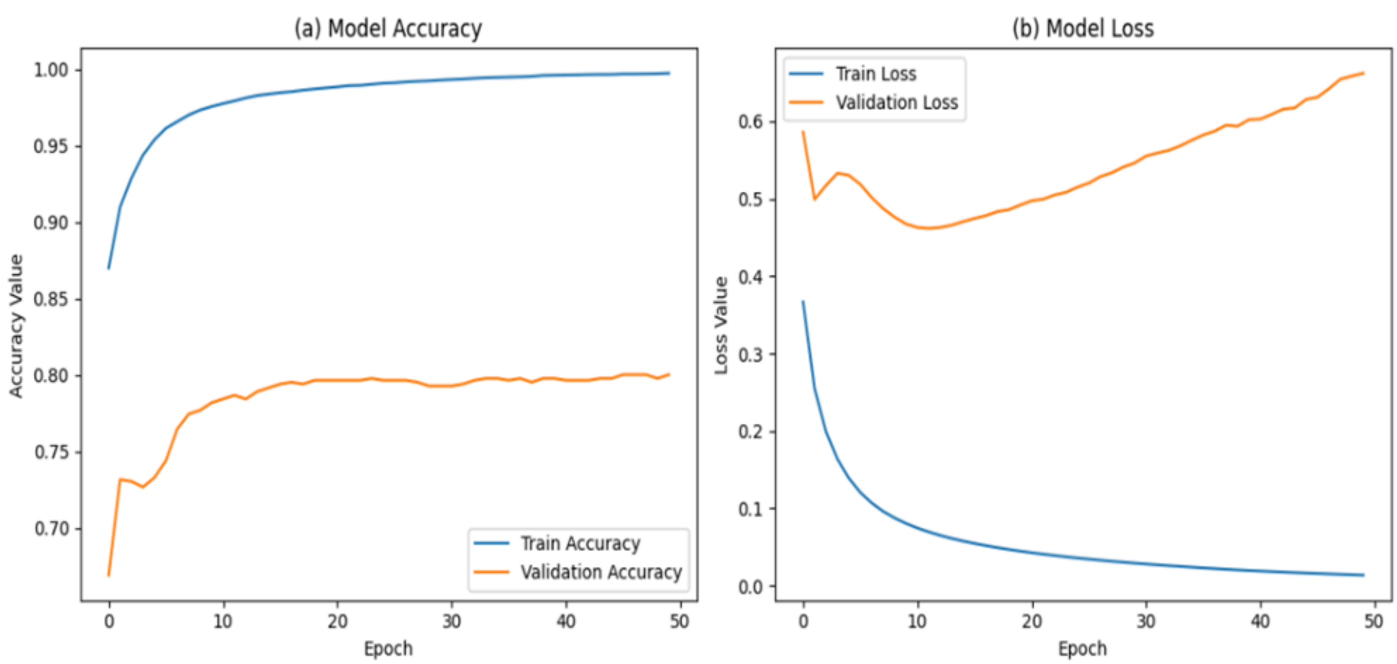


Figure X illustrates the training and validation performance of the MLP (Multilayer Perceptron) model across 50 epochs. Subfigure (a) shows the model accuracy over time, while subfigure (b) depicts the corresponding loss values.

As observed in Figure (a), the **training accuracy** increases steadily and reaches nearly 100%, suggesting that the model fits the training data very well. The **validation accuracy**, however, plateaus early and hovers around 80%, indicating limited improvement on unseen data and the possibility of overfitting.

Figure (b) supports this observation: while the **training loss** consistently decreases throughout the epochs, the **validation loss** initially decreases but then starts increasing after around epoch 10. This upward trend in validation loss, despite increasing accuracy on the training set, is a classic sign of overfitting—where the model's performance on new data begins to deteriorate despite improvements on the training set.

These findings align with the patterns discussed in Hamza et al. (Figure 4 in their IEEE paper), where a divergence between training and validation curves suggested a generalization gap. In our case, although the model reaches high accuracy on the training set, the rise in validation loss emphasizes the importance of regularization techniques such as dropout, early stopping, or data augmentation to enhance generalization.

IMPROVISATION 1

HYPERPARAMETER FINE-TUNING: In machine learning, hyperparameters are configuration settings that define how a model learns – such as depth of a decision tree, number of layers in a neural network, or the learning rate in gradient descent.

Hyperparameter tuning is the process of searching for the best combination of hyperparameters that maximize model performance on a validation set. It's like searching for the best "settings" that help your model learn efficiently and accurately.

TYPES OF HYPERPARAMETER SEARCH:

1. Grid Search – Tries all combinations in a manually defined grid of parameters.
2. Random Search – Randomly samples combinations in a search space. Works well for MLPs.
3. Bayesian Optimization – Uses past evaluation results to model performance landscape and predict better regions to explore next.

ATTEMPTS AT FINE TUNING OF MODELS:

RandomSearchCV optimisation for MLP (Multi-Layer Perceptron):

The MLP classifier was fine-tuned using RandomSearchCV. Instead of exhaustively trying all combinations (like GridSearchCV), RandomizedSearchCV samples a fixed number of random combinations. This is especially helpful when:

- You have limited compute resources
- You want a good result quickly

Training a neural network can take longer than a basic classifier like logistic regression. So it is often better to use randomness to explore hyperparameter space more efficiently.

Exploring the hyperparameters used for fine-tuning:

1. **'hidden_layer_sizes'** : [(128,), (64, 32), (128, 64, 32)] : Defines the architecture of the network. This affects the capacity and depth of the model.
2. **'activation'**: ['relu', 'tanh'] : Activation functions apply non-linearity. 'relu' is the default activation function and faster to train. Works well with deeper networks.
3. **'solver'**: ['adam'] : Solver is used for weights updates. 'adam' combines momentum + adaptive learning rates.
4. **'alpha'**: uniform(1e-5, 1e-2) : Controls strength of regularisation to prevent over-fitting.
5. **'learning_rate_init'**: Controls how big the steps are taken when updating weights. If it is too high it may overshoot minima and if too low, training is very slow.
6. **'batch_size'**: [32, 64, 128] : Affects training speed and durability. A larger batch has smoother updates but might take longer.

7. **'learning_rate'**: ['constant', 'adaptive'] : Letting the model choose whether dynamic adjustment helps convergence.
8. **'max_iter':** Maximum number of training iterations/epochs.

```
Best params: {'activation': 'relu', 'alpha': 0.0031362421095268924, 'batch_size': 32, 'hid
den_layer_sizes': (64, 32), 'learning_rate': 'constant', 'learning_rate_init': 0.003869320
9214799716, 'max_iter': 500, 'solver': 'adam'}
Test accuracy: 0.75
```

Potential reasons for lower accuracy scores:

1. MLPs Are Sensitive to Hyperparams:

- MLPs can easily underfit or overfit depending on the learning rate, network architecture, or regularization.
- A bad combination (e.g., large learning rate + small network + strong regularization) can hurt performance worse than the default.

2. Random Search Includes Bad Combinations:

- Random search explores a wide variety of hyperparameter combinations, including many suboptimal ones.
- Even with 50 iterations, it may not have found a good set yet — especially with so many dimensions.

3. Learning Rate + Alpha conflict:

- If both are high, underfitting may occur.
- If both are low, the model may overfit or be unstable.

Nevertheless, the MLP model consistently shows accuracy of over 75 %.

CONCLUSION:

As a part of our experiments to increase the accuracy of predictions regarding real/fake audio, we have explored hyper-parameter fine-tuning, its various methods and the results it gave when applied to a few standard machine learning models.

IMPROVISATION 2

DEEP LEARNING: Deep learning is a subfield of machine learning that automatically extracts patterns and characteristics from data using multi-layered neural networks. For tasks like image identification, audio analysis, and natural language understanding, it works incredibly well because it replicates the way the human brain processes information.

Deep learning models learn straight from raw data, in contrast to typical machine learning, which frequently requires manual feature extraction. These models are perfect for handling complicated issues in fields like deepfake detection, self-driving cars, and medical diagnosis since they become better with more data and processing capacity.

HOW DEEP LEARNING IS APPLIED HERE

In order to identify deepfake audio in complicated datasets, the study applies deep learning to MFCC-based spectrogram pictures using VGG-16 and LSTM models. Deep audio elements that conventional techniques could overlook are automatically learnt by these models.

Using a VGG-16 model that has been fine-tuned on spectrogram pictures of audio signals and loaded with ImageNet weights, the algorithm leverages deep learning. This improves the model's ability to distinguish between authentic and fraudulent audio.

A particular kind of deep learning model called LSTM (Long Short-Term Memory) is designed to comprehend time-series or sequential data, such as speech or audio signals. It is perfect for situations where context across time is important since it can recall long-term dependencies in the data.

In order to identify patterns that distinguish genuine speech from fraudulent speech over time, LSTM is utilised in this work to process MFCC features that have been collected from audio. Although it did well, VGG-16 did significantly better in terms of accuracy on the original dataset.

We have also fused together VGG-16 and LSTM for deepfake detection. By combining the advantages of both models, VGG-16 and LSTM enhance deepfake audio detection:

In order to distinguish between real and fake voices, VGG-16 extracts rich spatial characteristics from MFCC spectrogram images by identifying patterns in pitch, tone, and frequency.

After processing these features over time, LSTM learns the speech's temporal dynamics and flow, which deepfakes frequently can't naturally mimic.

This combination enables the algorithm to examine the sound's appearance as well as its changes over time, resulting in more reliable and accurate deepfake identification, particularly in complicated or noisy audio.

WHAT IS VGG-16 AND LSTM

The Visual Geometry Group at Oxford University created the deep convolutional neural network (CNN) architecture known as VGG-16.We convert 1D audio features (like MFCCs) to 2D spectrograms so that VGG-16, which is designed for image input, can process them. This allows it to learn spatial patterns in

frequency and time, similar to how it processes visual textures. In order to handle picture data, it has 16 layers, mostly convolutional layers followed by fully connected layers. Its ability to capture fine-grained patterns in images is a result of its use of small (3×3) convolutional filters with a great depth. Because of its straightforward yet efficient design, VGG-16 has found extensive application in picture categorisation, object identification, and transfer learning.
VGG-16 is used to learn the spatial patterns in the sound frequency in spectrogram images, which are visual representations of audio, in order to detect deepfake audio. Subtle changes and artefacts caused by artificial speech production techniques can be extracted by VGG-16 once MFCC characteristics have been transformed into these spectrograms. Compared to conventional methods, these learnt elements are subsequently utilised for classification, aiding in the more precise separation of authentic and fraudulent audio.

LSTM (Long Short-Term Memory) is a form of recurrent neural network (RNN) designed to handle sequential data by remembering long-term dependencies. In contrast to conventional RNNs, LSTMs regulate what data is remembered, transferred to the following time step, or stored using memory cells and gating mechanisms. Because of this, they work particularly well with time-series data, such as text, voice, and audio, where the sequence and length of the information are important.
LSTMs are used in deepfake audio detection to examine the temporal evolution of audio features, including MFCCs, across time. This makes it possible for the model to identify irregularities in speech patterns, tone, or rhythm that could point to artificial generation. LSTMs are especially helpful in identifying these tiny but telling abnormalities because deepfake audios frequently fail to accurately replicate natural speaking patterns.

RESULTS:

Similar to the paper, VGG-16 has outperformed LSTM here though the accuracies varied. We have implemented the VGG-16 and LSTM models for for-rerec datasets. We got quite satisfying results. The accuracies seem to reach a peak of near to 90% at first and then drop to some value.
The results are attached below in the results section.

Using models such as VGG-16 and LSTM increases deepfake detection efficiency by:
Automating feature extraction eliminates the need for manual feature engineering by learning pertinent patterns (such as pitch, tone shifts, and unnatural fluctuations) straight from the data.
Handling complexity: Unlike standard models, they do well on high-dimensional, loud, and varied-length audio.
Enhancing accuracy: By identifying minute, realistic-sounding alterations in phoney audio, VGG-16 and LSTM lower false positives and negatives.
Better scaling: These deep models are more effective for real-world applications where diverse audio is prevalent since they generalise well with huge datasets.

RESULT

ML ALGORITHMS:

Model Evaluation Results:

	Dataset	Model	Accuracy	Precision	Recall	F1 Score
0	rerec	Random Forest	0.697304	0.755717	0.697304	0.678971
1	rerec	SVM	0.781863	0.844524	0.781863	0.771472
2	rerec	MLP	0.762255	0.811339	0.762255	0.752500
3	rerec	XGBoost	0.741422	0.795850	0.741422	0.728955

HYPERPARAMETER FINE TUNING:

```
Fitting 3 folds for each of 30 candidates, totalling 90 fits
Best params: {'activation': 'relu', 'alpha': 0.0006588416215095006, 'batch_size': 128, 'hidden_layer_sizes': (128,
64, 32), 'learning_rate': 'constant', 'learning_rate_init': 0.008752168275567039, 'max_iter': 500, 'solver': 'ada
m'}
Test accuracy: 0.8590686274509803
```

VGG 16 AND LSTM::

```
313
Epoch 19/20
319/319 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9197 - loss: 0.2019
Epoch 19: val_accuracy improved from 0.90241 to 0.91043, saving model to rerec_vgg16_model.keras
319/319 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step - accuracy: 0.9197 - loss: 0.2019 - val_accuracy: 0.9104 - val_loss: 0.2
346
Epoch 20/20
319/319 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9176 - loss: 0.1898
Epoch 20: val_accuracy did not improve from 0.91043
319/319 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step - accuracy: 0.9176 - loss: 0.1899 - val_accuracy: 0.8899 - val_loss: 0.2
359
```

```
Training LSTM model...
Epoch 1/20
317/319 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5982 - loss: 0.6454
Epoch 1: val_accuracy improved from -inf to 0.78209, saving model to rerec_lstm_model.keras
319/319 ━━━━━━━━━━━━━━━━━━━━ 8s 12ms/step - accuracy: 0.5991 - loss: 0.6447 - val_accuracy: 0.7821 - val_loss: 0.4
743
Epoch 2/20
319/319 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8100 - loss: 0.4283
Epoch 2: val_accuracy improved from 0.78209 to 0.85740, saving model to rerec_lstm_model.keras
319/319 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8100 - loss: 0.4282 - val_accuracy: 0.8574 - val_loss: 0.3
300
Epoch 3/20
319/319 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8544 - loss: 0.3393
Epoch 3: val_accuracy did not improve from 0.85740
319/319 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8545 - loss: 0.3392 - val_accuracy: 0.8507 - val_loss: 0.4
232
Epoch 4/20
318/319 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8661 - loss: 0.3209
Epoch 4: val_accuracy improved from 0.85740 to 0.89750, saving model to rerec_lstm_model.keras
319/319 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.8662 - loss: 0.3207 - val_accuracy: 0.8975 - val_loss: 0.2
594
Epoch 5/20
319/319 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9071 - loss: 0.2412
Epoch 5: val_accuracy did not improve from 0.89750
319/319 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9071 - loss: 0.2412 - val_accuracy: 0.8311 - val_loss: 0.3
684
```

```
Final results:

Dataset: rerec
  VGG16 model:
    accuracy: 0.8799
    precision: 0.8817
    recall: 0.8799
    f1: 0.8798
  LSTM model:
    accuracy: 0.5515
    precision: 0.5525
    recall: 0.5515
    f1: 0.5492
```

+ Code + Markdown

CONCLUSION

In this project, we successfully developed a machine learning and deep learning pipeline for the detection of deepfake audio, leveraging MFCC-based feature engineering and spectrogram analysis. Using the Fake-or-Real (FoR) dataset, we explored various machine learning models such as SVM, Random Forest, MLP, and XGBoost, and fine-tuned hyperparameters to enhance performance. We further implemented a deep learning approach by fine-tuning a VGG-16 model on MFCC spectrograms, achieving notable improvements, especially on challenging datasets.

The results demonstrate that MFCC features combined with traditional classifiers can effectively distinguish real from synthetic audio. Additionally, deep learning models show promise for handling more complex and heterogeneous datasets. Visual analysis of spectrograms and MFCCs validated the presence of distinguishing artifacts between real and fake audio samples.

Through hyperparameter tuning and transfer learning, we highlighted the importance of model optimization for better generalization. This project lays a solid foundation for future work in deepfake audio detection, suggesting directions such as the use of Transformer-based architectures and expanding datasets with more real-world noisy samples to improve robustness and scalability.

Overall, our work provides a comprehensive and practical solution to the emerging threat of deepfake audio, contributing meaningfully to the domain of audio forensics and digital security.

REFERENCES

1. A. Hamza et al., "Deepfake Audio Detection via MFCC Features Using Machine Learning," in IEEE Access, vol. 10, pp. 134018-134028, 2022, doi: 10.1109/ACCESS.2022.3231480.

2. Rana, Md & Murali, Beddhu & Sung, Andrew. (2021). Deepfake Detection Using Machine Learning Algorithms. 10.1109/IIAI-AAI53430.2021.00079.

3. A. Qais, A. Rastogi, A. Saxena, A. Rana and D. Sinha, "Deepfake Audio Detection with Neural Networks Using Audio Features," 2022 International Conference on Intelligent Controller and Computing for Smart Power (ICICCSP), Hyderabad, India, 2022, pp. 1-6, doi: 10.1109/ICICCSP53532.2022.9862519.

4. Iqbal, F., Abbasi, A., Javed, A. R., Jalil, Z., & Al-Karaki, J. (2022). Deepfake Audio Detection via Feature Engineering and Machine Learning. *CEUR Workshop Proceedings*, *3318*.

5. Priya N V, Pavan H, Prajwal S, Varun R, Vinay A., Deepfake Audio Detection using MFCC Features. IJFMR250136558

CONTRIBUTION

Research Paper Implementation - Anvay Sawai
Visualization - Shashwat Pandey
Hyperparameter Fine Tuning - Pradyumna Udupa
Deep Learning - M Hrushikesh